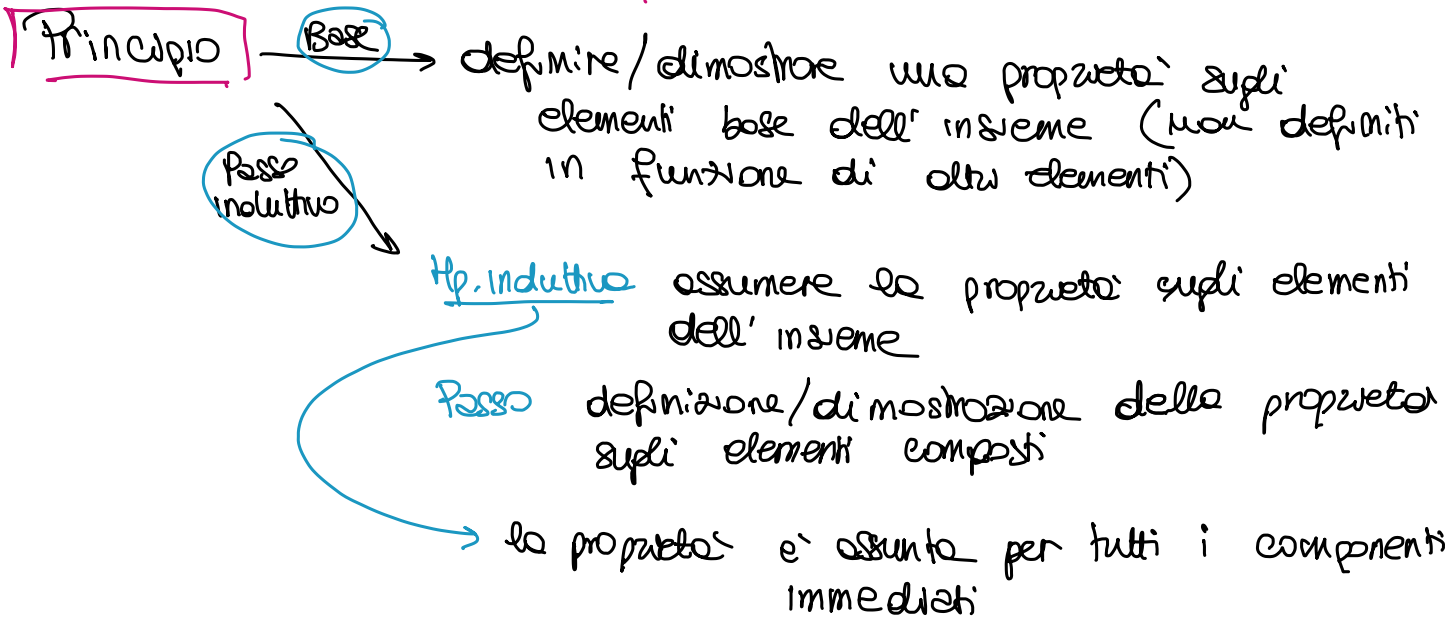


INDUZIONE STRUTTURALE



$X \subseteq \mathbb{N}$ Def:

Base : $0 \in X$

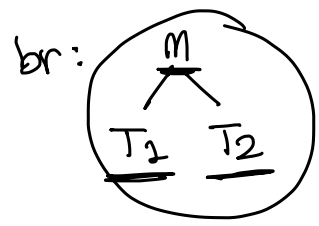
Passo : se $x \in X$ allora $x+3 \in X$

} dim. $X = 3\mathbb{N}$

Alberi binari BT Nodi N

Base : $m \in \mathbb{N}$ è un BT ($m \in BT$) .. $\mathbb{N} \subseteq BT$

Passo : se T_1 e $T_2 \in BT$ allora $br(m, T_1, T_2) \in BT$



Foglie: conta il numero di foglie di $T \in BT$

$$\begin{cases} \text{foglie}(m) = 1 \\ \text{foglie}(br(m, T_1, T_2)) = \text{foglie}(T_1) + \text{foglie}(T_2) \leftarrow \end{cases}$$

Nodi: conta il numero di nodi di $T \in BT$ $\begin{cases} \text{nodi}(m) = 1 \\ \text{nodi}(br(m, T_1, T_2)) = \text{nodi}(T_1) + \text{nodi}(T_2) + 1 \end{cases}$

Branch: conta il numero di rami di $T \in BT$

$$\begin{cases} \text{branch}(m) = 0 \\ \text{branch}(\text{br}(m, T_1, T_2)) = \underline{\text{branch}(T_1) + \text{branch}(T_2) + 1} \end{cases}$$

$$\forall T \in BT \quad \text{foglie}(T) = \text{branch}(T) + 1$$

Base $m \in N \quad \text{foglie}(m) = 1 + 0 = 1 + \text{branch}(m) \quad \checkmark$
 $\text{branch}(m) = 0$

Passo $\text{br}(m, T_1, T_2) \in BT \quad T_1, T_2 \in BT$
 $\boxed{\text{Hp. ind}} \rightarrow \text{assumere la proprietà per i componenti immediati}$

$$\text{foglie}(T_1) = \text{branch}(T_1) + 1$$

$$\text{foglie}(T_2) = \text{branch}(T_2) + 1$$

$$\underline{\text{foglie}(\text{br}(m, T_1, T_2))} = \text{foglie}(T_1) + \text{foglie}(T_2)$$

per def foglie

$$= \text{branch}(T_1) + 1 + \text{branch}(T_2) + 1$$

Hp. ind

$$= (\text{branch}(T_1) + \text{branch}(T_2) + 1) + 1$$

$$= \underline{\text{branch}(\text{br}(m, T_1, T_2))} + 1 \quad \checkmark$$

$$\textcircled{P} \rightarrow az \mid bb \mid \textcircled{bP} \mid \textcircled{aPa}$$

$$\Sigma = \{a, b\}^*$$

def. insieme SP : $az, bb \in SP$

$\& x \in SP$ allora $axa \in SP$
 $bxb \in SP$

Dimostrare: $\forall x \in SP \quad \exists yz \in \{a,b\}^* \quad x = yz \text{ e } y = \text{reverse}(z)$

Base: $x = aa, \quad y = a, \quad z = a$

$\Downarrow$

$$x = yz = aa$$

$$y = \text{reverse}(z) = a$$



$x = bb, \quad y = b, \quad z = b$

$\Downarrow$

$$x = yz = bb$$

$$y = \text{reverse}(z) = b$$

Passo: $x \in SP$

$\hookrightarrow a \underline{x} a \stackrel{x'}{=}$

Ip. ind. $\exists yz. \underline{x = yz} \quad y = \text{reverse}(z)$

dobbiamo dim. $\exists y'z'. x' = aza$

$$\text{reverse}(z') = y'$$

$$y' = ay \quad z' = za \Rightarrow x' = aza = \underbrace{ay}_{\text{def } x} z a$$

$$= y' z'$$

$$\text{reverse}(z') = \text{reverse}(za) \stackrel{\text{def reverse}}{=} a \text{reverse}(z)$$

$$= ay = y' \stackrel{\text{Ip ind}}{=} y' \stackrel{\text{def } y'}{=}$$

$\hookrightarrow b a b$ analogo

Grammatica espressioni aritmetiche

$$E \rightarrow m \mid E + E \mid E * E \quad m \in \mathbb{N}$$

Base $m \in \text{Exp} \quad (m \in \mathbb{N})$

Passo se $e_1, e_2 \in \text{Exp}$ allora

$$e_1 + e_2 \in \text{Exp}$$

$$e_1 * e_2 \in \text{Exp}$$

op: conta il numero di operatori in una espressione

$$\text{op}(m) = 0$$

$$\text{op}(e_1 + e_2) = \text{op}(e_1 * e_2) = \text{op}(e_1) + \text{op}(e_2) + 1$$

val: conta il numero di valori in una espressione

$$\text{val}(m) = 1$$

$$\text{val}(e_1 + e_2) = \text{val}(e_1 * e_2) = \text{val}(e_1) + \text{val}(e_2)$$

Dimostriamo $\forall e \in \text{Exp} \quad \text{val}(e) = \text{op}(e) + 1$

Base $m \in \text{Exp} \quad \text{val}(m) = 1 = 1 + 0 = 1 + \text{op}(m) \quad \checkmark$

Passo $e_1, e_2 \in \text{Exp} \quad e_1 + e_2$ (analogo $e_1 * e_2$)
dobbiamo dim. $\text{val}(e_1 + e_2) = \text{op}(e_1 + e_2) + 1$

Hp. ind $\begin{cases} \text{val}(e_1) = \text{op}(e_1) + 1 \\ \text{val}(e_2) = \text{op}(e_2) + 1 \end{cases}$

$$\text{val}(e_1 + e_2) \underset{\text{def val}}{=} \underset{\text{Hp. ind}}{\text{val}(e_1)} + \underset{\text{Hp. ind}}{\text{val}(e_2)} = \text{op}(e_1) + 1 + \text{op}(e_2) + 1$$

$$= (\text{op}(e_1) + \text{op}(e_2) + 1) + 1$$

$$\underset{\text{def op}}{=} \text{op}(e_1 + e_2) + 1$$

COMPOSIZIONALITA'

Il significato di ogni costrutto deve essere funzione del significato delle componenti immediate.

SISTEMI DI TRANSIZIONE

- Matematicamente precisi
- concisi
- Permettono di dare semantica in modo induttivo sulla struttura della sintassi
- Questo avviene mediante un insieme di regole
- Metodo di specifica generale → permette astrazione

Def

$\langle T, \rightarrow \rangle$ è un sistema di transizione dove

T insieme di configurazioni / stati $\gamma \in T$
(descrizione formale dello stato della macchina)

→ relazione binaria di transizione

$$\rightarrow \subseteq T \times T$$

Notazioni $(\gamma_1 \gamma_2) \in \rightarrow$ viene denotato $\gamma_1 \rightarrow \gamma_2$

$$\gamma_0 \rightarrow^* \gamma_n \Leftrightarrow \exists \gamma_1 \gamma_2 \dots \gamma_{n-1} \text{ t.c.}$$

$$\gamma_0 \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \dots \rightarrow \gamma_{n-1} \rightarrow \gamma_n$$

Sistema di transizione deterministico: $\langle T, T, \rightarrow \rangle$ dove

$\langle T, \rightarrow \rangle$ è un sistema di transizione

$T \subseteq T$ insieme di configurazioni deterministici

t.c. $\forall \gamma \in T. \nexists \gamma' \in T$
 $\gamma \rightarrow \gamma'$

Grammatica $G = \langle V, T, P, S \rangle \rightarrow \langle T, \Rightarrow^T \rangle$ Sistema di transizione

$$T = P$$

$$T = A \rightarrow \alpha$$

$$\uparrow$$

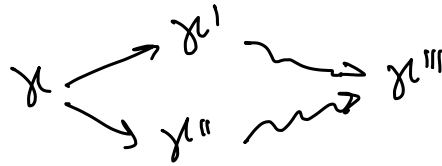
$$T^*$$

$$\frac{A \rightarrow \alpha}{\beta A \gamma \rightarrow \beta \alpha \gamma}$$

$$\equiv (A \rightarrow \alpha) \rightarrow (\beta A \gamma \rightarrow \beta \alpha \gamma)$$

$$S \rightarrow \alpha \in T^*$$

Non determinismo



$$e_1 + e_2$$

CATEGORIE SINTATTICHE

↳ manipolare dati (espressioni)

↳ creazione/manipolazione di legami (dichiarazioni)

↳ trasformare stati (comandi)

Ambiente

Insieme dei legami
 $Id \leftrightarrow oggetti$

Memoria

permette di descrivere gli effetti di una trasformazione

ESPRESSIONI

denotano valori

vengono valutate per restituire un valore

Sono equivalenti se denotano lo stesso valore

(Sono uguali & sono sintatticamente uguali)

$$6 * 1$$

$$2 * 3$$

$$4 + 2$$

DICHIARAZIONI

→ Denotano richieste di modifica degli ambienti

→ vengono elaborate per attuare le trasformazioni dell'ambiente

↓
dichiarazioni equivalenti se vengono elaborate nello stesso ambiente

COMANDI

→ Denotano richieste di modifica della memoria

→ vengono eseguiti per ottenere la trasformazione della memoria corrispondente

↓
programmi equivalenti generano lo stesso input e stesse trasformazioni della memoria.